

Multi-Domain 6G Network Slice Orchestration

Part II — Proposed Architecture

Two-Agent LLM Generator · MAPPO under Dec-POMDP · MILP Validator · RL Feedback Loop

1 Design Decisions and Rationale

Single operator, multi-domain. All M domains belong to one administrative authority. Because all domains are under one operator, the digital twin can simulate the full joint state s_t during training — used exclusively by the centralized critic, never by the domain actors. This enables clean CTDE without federated learning. The multi-operator extension is deferred as future work.

Dec-POMDP formulation with MAPPO. The placement problem is formalised as a *Decentralized Partially Observable MDP* (Dec-POMDP). Each domain m has its own actor π_θ^m trained on its local observation o_t^m only — both during training and at inference. A single shared centralized critic $V_\phi(s_t)$ accesses the full joint state during training only and is discarded at inference. This is genuine CTDE: the observation seen by each actor at inference is identical to what it saw during training, eliminating the observation gap that a single global actor would suffer from.

Two-agent LLM Generator. Module 1 is structured as two sequential LLM agents. **Agent A** (Intent Translator) converts a natural-language service intent into a fully specified technical slice request. **Agent B** (Plan Proposer) takes that specification plus the serialised infrastructure state and proposes K concrete candidate plans. Agent A operates zero-shot; Agent B uses few-shot prompting with examples drawn from MILP solutions.

RL feedback to Agent B (closed-loop enrichment). After each training episode, signals derived from the domain actors are aggregated and fed back into Agent B’s prompt context. These signals include: score margins of previously proposed plans, and constraint identifiers that are routinely binding across episodes. This conditions Agent B to propose plans that are more likely to score highly and to avoid systematically infeasible configurations.

Quality-score action space. Each domain actor outputs a quality score vector $\mathbf{q}^m \in [0, 1]^K$ over the K candidate plan fragments, rather than a binary accept/reject vote. The MDO aggregates scores across domains and selects the highest-scoring structurally valid plan.

MILP as oracle during training; LP check at inference. The full MILP is solved on sampled static batches during training to produce z_{MILP}^* , which serves as an offline evaluation benchmark. At inference, all binary variables are fixed by the actors’ selections and the MILP Validator performs only a polynomial-time linear feasibility check on constraints C2, C3, C5b, C7.

2 Formal Model — Dec-POMDP

The multi-domain placement problem under dynamic arrivals is cast as a Dec-POMDP $(\mathcal{S}, \{O^m\}_{m=1}^M, \{\mathcal{A}^m\}_{m=1}^M, \mathcal{P}, \mathcal{R}, \gamma_{\text{RL}})$.

2.1 Global State

The global state $s_t \in \mathcal{S}$ is accessible *only* to the centralized critic during training:

$$s_t = \left(\{c_n^{\text{res}}, r_n^{\text{res}}\}_{n \in \mathcal{N}}, \{b_\ell^{\text{res}}\}_{\ell \in \mathcal{L}}, \{|\mathcal{S}_{\text{act}}^m|\}_{m=1}^M, \{u_\ell\}_{\ell \in \mathcal{L}^{\text{inter}}}, s_{\text{req},t} \right), \quad (1)$$

where $c_n^{\text{res}}, r_n^{\text{res}}$ are exact per-node residual capacities, b_ℓ^{res} is residual link bandwidth, $|\mathcal{S}_{\text{act}}^m|$ is the active slice count in domain m , u_ℓ is inter-domain link utilisation, and $s_{\text{req},t}$ is the arriving slice request.

2.2 Local Observation

Each domain agent m observes only its local state:

$$o_t^m = O(s_t, m) = \left(\{c_n^{\text{res}}, r_n^{\text{res}}\}_{n \in \mathcal{N}^m}, \{b_\ell^{\text{res}}\}_{\ell \in \mathcal{L}^m}, \pi_{\text{cand},t}^m \right). \quad (2)$$

Here $\pi_{\text{cand},t}^m$ is the portion of the MDO candidate plan assigned to domain m

2.3 Action Space

Each domain actor m observes its assigned fragment π_i^m against its local state o_t^m and outputs a quality score over the K candidate fragments:

$$\mathcal{A}^m = [0, 1]^K, \quad \mathbf{q}^m = \pi_\theta^m(o_t^m) \in [0, 1]^K, \quad (3)$$

where $q_i^m \in [0, 1]$ is actor m 's quality score for fragment π_i^m . The score encodes residual slack, load distribution, and the likelihood that the fragment contributes to a globally feasible and efficient plan.

The MDO coordinator aggregates scores and selects the highest-scoring structurally valid plan:

$$J_i = \sum_{m=1}^M q_i^m, \quad \pi^* = \arg \max_i J_i, \quad (4)$$

If no plan passes the structural check, Agent B is invoked again with the violation details appended to its prompt.

2.4 Reward Function

A shared global reward is broadcast to all domain actors after each slice arrival event. The digital twin acts as the environment: when the selected plan π^* is executed, the twin updates its state (reserving CPU, RAM, and bandwidth on the relevant nodes and links) and evaluates whether all constraints are satisfied in the resulting state. The reward is computed from this state transition:

$$\mathcal{R}_t = \underbrace{\mu \cdot z_s}_{\text{admission}} - \underbrace{\alpha \cdot \text{Cost}(\pi^*)}_{\text{efficiency}} - \underbrace{\lambda_{\text{viol}} \cdot \mathbf{1}[\text{Constraints violated in twin}]}_{\text{hard penalty}} + \underbrace{\eta \cdot \text{LocalScore}(\pi^*)}_{\text{quality shaping}} + \underbrace{\kappa \cdot \text{Margin}(\mathbf{J})}_{\text{dense shaping}} \quad (5)$$

where $\mu \gg \alpha, \eta, \kappa$ ensures admission always takes priority. The terms are defined as follows.

Admission ($\mu \cdot z_s$). $z_s \in \{0, 1\}$ indicates whether the slice was admitted. This is the primary objective. μ is set large enough to dominate all other terms, so the agent’s first priority is always to find a valid plan and admit the slice.

Efficiency ($-\alpha \cdot \text{Cost}(\pi^*)$). $\text{Cost}(\pi^*)$ aggregates total CPU, RAM, and bandwidth consumed by the selected plan. The agent is penalised for over-provisioning and rewarded for using the minimum resources necessary.

Hard penalty ($-\lambda_{\text{viol}} \cdot \mathbf{1}[\cdot]$). This term fires when the digital twin’s state transition reveals that the executed plan violates C2 (CPU), C3 (RAM), C5b (throughput), or C7 (E2E delay). Constraints C1, C4, C6, and C8 are guaranteed upstream by Agent B’s structural checker and never violated at this stage. Over training episodes, actors learn to score fragments lower when local resource states suggest downstream violations are likely observation.

Quality shaping ($+\eta \cdot \text{LocalScore}(\pi^*)$).

$$\text{LocalScore}(\pi^*) = \frac{\text{Cost}_{\text{greedy}} - \text{Cost}(\pi^*)}{\text{Cost}_{\text{greedy}}} \in [0, 1], \quad (6)$$

where $\text{Cost}_{\text{greedy}}$ is the cost of a greedy first-fit placement for the same arriving request, computable in $O(|\mathcal{F}_s| \cdot |\mathcal{N}|)$. This term provides a dense, per-request quality signal that fires every timestep regardless of whether the slice is admitted, addressing the sparsity of the admission signal alone.

Dense shaping ($+\kappa \cdot \text{Margin}(\mathbf{J})$).

$$\text{Margin}(\mathbf{J}) = J_{i^*} - \frac{1}{K-1} \sum_{i \neq i^*} J_i \in [-1, 1]. \quad (7)$$

This rewards actors for being decisive in their scoring, assigning a clearly higher joint score to one plan over the alternatives. It discourages score collapse (all K plans receiving nearly identical scores), which would make plan selection arbitrary. Like LocalScore , it fires every timestep.

3 Module 1 — Two-Agent LLM Generator

3.1 Agent A — Intent Translator

Agent A receives a natural-language service intent and outputs a complete technical slice request $s = (\mathcal{F}_s, \mathcal{E}_s, \mathbf{q}_s, \mathcal{D}_s, \tilde{c}_{f_k^s}, \tilde{r}_{f_k^s})$. It operates zero-shot: the mapping from service type to SFC structure is well covered by LLM pre-training on 5G/6G standards. Agent A is not re-invoked during the self-correction loop — its specification is preserved across retries.

3.2 Agent B — Plan Proposer

Agent B receives four inputs:

1. Technical slice request s from Agent A.
2. Serialised substrate state: per-node residual fractions ($c_n^{\text{res}}/C_n, r_n^{\text{res}}/R_n$), per-link residual bandwidth, active slice count per domain.
3. Few-shot examples: 3–5 triples (substrate state, slice request, MILP-optimal plan) drawn from training instances. Zero-shot is evaluated as an explicit ablation.
4. Feedback context: score margins of recent proposals, binding constraint identifiers, and violation messages from the structural checker or MILP Validator. This guides Agent B away from configurations that consistently fail, without any weight updates.

Each candidate plan π_i is a JSON object containing: placement ($f_k^s \mapsto n_k$), allocation ($f_k^s \mapsto (a_k^c, a_k^r)$), routing (ordered link sequence per flow), and bandwidth (a_k^b per flow).

3.3 Structural Constraint Checker

Before any plan reaches Module 2, a local checker verifies C1, C4, C6, and C8 in $O(|\mathcal{F}_s| \cdot |\mathcal{N}| + |\mathcal{E}_s| \cdot |\mathcal{L}|)$ time without any LP solve. Violated plans are discarded and their violations appended to Agent B’s prompt for regeneration.

4 Module 2 — RL Selector (MAPPO, Dec-POMDP)

4.1 Architecture

Module 2 implements MAPPO under the Dec-POMDP formulation:

- M domain actors $\{\pi_\theta^m\}_{m=1}^M$, each taking local observation o_t^m as input and outputting a quality score vector $\mathbf{q}^m \in [0, 1]^K$. Actors share a backbone network but have separate output heads.
- One shared centralized critic $V_\phi(s_t)$ taking the full global state during training. Discarded at inference.

4.2 CTDE Training Procedure

1. **Initialisation.** The digital twin instantiates a random multi-domain substrate. Each actor π_θ^m is initialised with shared weights.
2. **Episode rollout.** Slice requests arrive (Poisson, rate λ). For each arrival: Agent B generates K plans; each actor m observes o_t^m and outputs $\mathbf{q}^m$; the MDO aggregates scores and selects π^* ; the environment transitions; the global reward $\mathcal{R}_t$ is broadcast to all actors.
3. **Advantage estimation.** The shared critic computes $V_\phi(s_t)$ from the full joint state. Per-agent advantages $A_t^m = \mathcal{R}_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ are used to update each actor independently via the PPO clipped surrogate objective, with entropy regularisation on $\mathbf{q}^m$ to prevent score collapse.
4. **RL feedback extraction.** After each episode, score margins and binding constraint identifiers are aggregated and fed back into Agent B’s prompt context for the next episode.

4.3 Deployment

After training, the critic is discarded. Each actor π_θ^m is deployed at domain m and runs using only its local state:

- receives a plan fragment from the MDO coordinator,
- evaluates it against its local observation o_t^m ,
- outputs a quality score q_i^m in a single forward pass (< 1 ms),
- reports only scalar aggregated summaries to the MDO.

No raw per-node telemetry crosses domain boundaries at any point.

5 Module 3 — MILP Validator

5.1 Inference Mode — Feasibility Certificate

With all binary variables fixed by the actors’ joint selection, the validator solves a linear program checking C2 (CPU), C3 (RAM), C5b (throughput floor), and C7 (E2E delay) in polynomial time. C1, C4, C6, C8 are already guaranteed by Agent B’s structural checker.

- **Feasible:** issue certificate $(s, \pi_{\text{sel}}, \checkmark)$ and execute.
- **Infeasible:** identify violated constraints and offending resources; serialise into a violation message; feed back to Agent B. New generation round from Module 1B. Agent A’s specification is preserved.

5.2 Evaluation Mode — MILP Oracle

Independently of the online training loop, Module 3 periodically solves the full MILP on sampled static batches to produce z_{MILP}^* . This value is used exclusively as an *offline*

evaluation instrument: it populates the optimality gap metric $1 - f(\pi_{\text{sel}})/z_{\text{MILP}}^*$ reported in Section ?? and validates how close the hybrid system approaches the theoretical upper bound. It has no effect on actor weights, reward signals, or Agent B’s prompt — the online training loop runs independently of it. The full MILP solve never occurs during inference.

6 Training vs. Inference — CTDE Summary

Table 1 summarises the information available to each component during training versus inference.

Table 1: Information access: training vs. inference.

Component	Training	Inference
Centralized critic V_ϕ	Full joint state s_t (all domains)	<i>Discarded</i>
Domain actor π_θ^m	Local obs. o_t^m only	Local obs. o_t^m only
Agent B (LLM)	State + few-shot + RL & validator feedback	State + few-shot
MILP Validator	Full solve (offline evaluation)	LP feasibility check only
MDO	Scalar aggregates from all domains	Scalar aggregates from all domains

The key property: **each actor observes the same information during training and inference**. There is no observation gap.

Evaluation metrics:

- Slice admission rate (%)
- Resource utilisation efficiency (%)
- E2E delay satisfaction rate (%)
- Optimality gap: $1 - f(\pi_{\text{sel}})/z_{\text{MILP}}^*$, where z_{MILP}^* is obtained from the offline MILP oracle on held-out static batches.
- Plan validity rate on first attempt (ablation: few-shot vs. zero-shot)
- Decision latency (ms, inference only)

References

1. C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24611–24624, 2022.
2. V. Onaji, C. Eze, and A. Okafor, “Deep Reinforcement Learning for Dynamic Network Slicing and Resource Orchestration in Software-Defined Networks,” *International Journal of Computer Applications Technology and Research (IJCATR)*, vol. 14, no. 11, pp. 53–73, 2025. DOI: 10.7753/IJCATR1411.1006.

3. P. Jamshidi, S. M. Hosseini, M. Mahdudi, M. A. Zamani, and M. R. Shakournia, "Multi-Domain Network Service Orchestration Framework: A Hierarchical Multi-Agent Federated Deep Reinforcement Learning Approach," SSRN Preprint 5143341, 2025..
4. A. Dandoush, V. Kumarskandpriya, M. Uddin, and U. Khalil, "Large Language Models meet Network Slicing Management and Orchestration," , 2024.
5. S. Nam, N. Van Tu, and J. W.-K. Hong, "LLM-Based AI Agent for VNF Deployment in OpenStack Environment," in *Proc. IEEE Network Operations and Management Symposium (NOMS)*, 2025.
6. J. Su, Y. Zhang, and X. Chen, "Leveraging Large Language Models for VNF Resource Forecasting in Network Slicing," in *Proc. IEEE International Conference on Network Softwarization (NetSoft)*, 2024.
7. C. Pham, N. H. Tran, S. Ren, W. Saad, and C. S. Hong, "Traffic-Aware and Energy-Efficient VNF Placement for Service Chaining: Joint Sampling and Matching Approach," *IEEE Transactions on Services Computing*, vol. 13, no. 1, pp. 172–185, Jan./Feb. 2020.

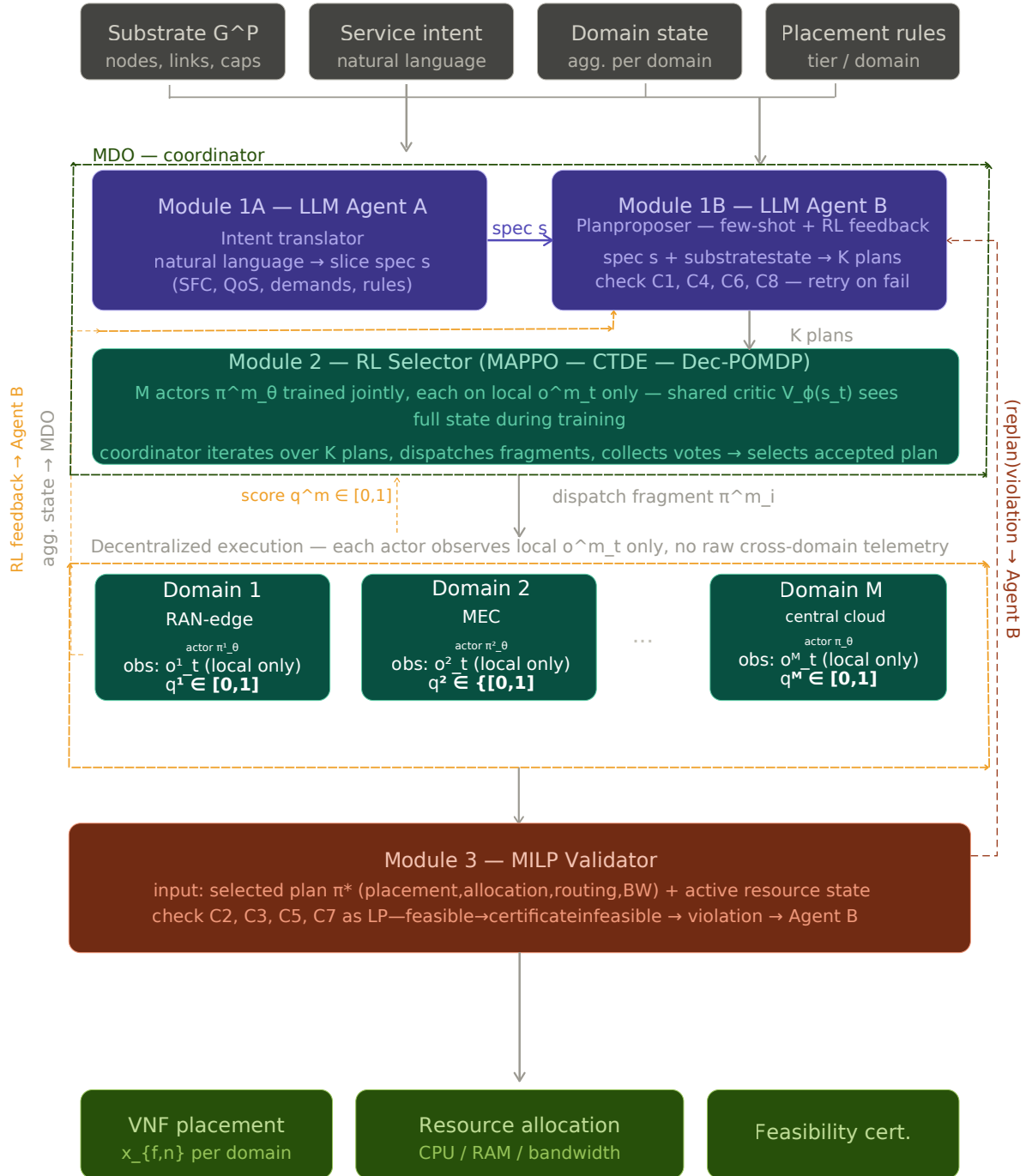


Figure 1: System Design V4